CNN-Based Weather Classification

Titly Bhattacharjee , Shagata Chowdhury, Arpita Das

0222210005101004, 0222210005101009, 0222210005101011 .
Neural Network and Fuzzy Logic Laboratory .
Date: November 23, 2025 .

Abstract

Accurate and automated weather classification from visual data is a significant task with applications in climate monitoring, agriculture, and autonomous systems. This project addresses the problem of classifying weather conditions from images by implementing a Convolutional Neural Network (CNN) from scratch using TensorFlow and Keras. The model is designed to distinguish between four distinct meteorological classes: 'cloudy,' 'rain,' 'shine,' and 'sunrise". The dataset, structured into training 900 images and validation 225 images sets, was loaded using Keras Image-Data-Generator , with extensive data augmentation techniques (rotation, shifts, shear, zoom, flip, brightness adjustment) applied to the training data to improve model generalization and combat overfitting. The proposed CNN architecture comprises four convolutional blocks, each followed by batch normalization, max-pooling, and dropout layers for feature extraction and regularization. This is succeeded by a classifier head with two dense layers. The model was compiled with the Adam optimizer and trained for 50 epochs . To ensure optimal training, callbacks for Early Stopping, Learning Rate Reduction, and Model Checkpointing were employed. The training process was highly successful, with the best model achieving a remarkable 96.00 percentage accuracy on the validation set . The final model and its training history were saved for future use. This project demonstrates the efficacy of a well-designed, custom CNN model combined with robust data preprocessing and training strategies for the task of image-based weather classification, achieving state of the art performance on the given dataset .

Keywords: CNN, Weather Classification, Image Data Augmentation

1 Introduction and Problem Statement

The ability to accurately identify weather conditions is fundamental to numerous human activities and technological systems . From planning daily commutes to enabling advanced meteorological research and ensuring the safety of autonomous vehicles, reliable weather recognition is crucial. Traditionally, this has been the domain of human observation and specialized sensor equipment. However, the proliferation of digital cameras and the advent of powerful computer vision techniques have opened up new avenues for automating this process using visual data. The core problem this project tackles is multi-class weather classification from static images. Given an input image, the goal is to correctly assign it to one of several predefined weather categories. Automating this task presents a compelling alternative or supplement to sensor-based methods , as it can leverage existing infrastructure like traffic cameras, smartphones, and satellites.

Challenges in Weather Image Classification:

- Intra-class Variability: A "cloudy" sky can range from thin, wispy cirrus to thick, dark cumulonimbus clouds. A "rainy" scene might show heavy downpours or just a damp road .
- Inter-class Similarity: A 'sunrise' scene can often appear similar to a 'shine' (sunset) scene in terms of color palette. A heavily overcast 'cloudy' day might be visually confused with a dim, overcast 'rainy' day .
- Varying Backgrounds and Context: The model must learn to focus on meteorological features (sky, clouds, light, precipitation) while ignoring irrelevant background objects like buildings, trees, and terrain .
- Limited Data: Collecting and labeling a large, high-quality dataset of weather images can be time-consuming and expensive, making the model susceptible to overfitting .

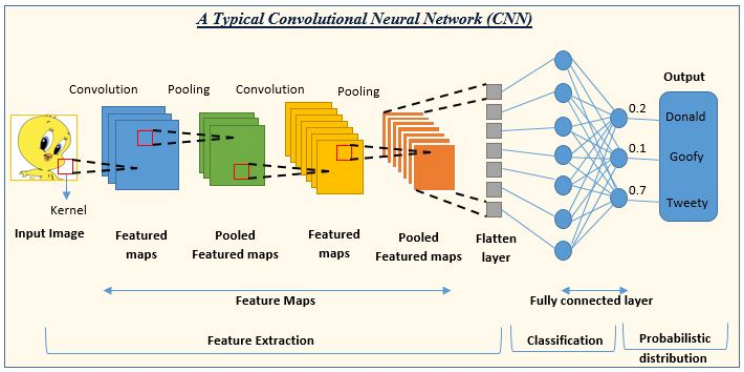

Fig. 1 A Typical Convolutional Neural Network

2 Related Work

The application of deep learning, particularly CNNs, to image classification has been a dominant trend in computer vision since the groundbreaking success of AlexNet in the 2012 ImageNet competition . The approach taken in this project builds upon well-established methodologies in this field . The model design in this project is inspired by the classic CNN pattern of stacking convolutional and pooling layers . While it does not use a specific pre-defined architecture like VGG, ResNet, or Inception, it follows their core principles . The use of successive convolutional blocks with increasing filter sizes (32, 64, 128, 256) allows the network to learn features at different levels of abstraction , from simple edges and textures in the initial layers to complex patterns and objects in the deeper layers . This hierarchical feature learning is a cornerstone of modern CNNs .The extensive use of data augmentation, as seen in the Image-Data-Generator configuration ,is a standard and critical technique to prevent overfitting, especially with limited datasets .By applying random transformations (rotations, flips, zooms, brightness changes)that are plausible in real-world images, the model is exposed to a much wider variety of training examples without collecting new data. This practice forces the model to learn more invariant features, significantly improving its ability to generalize to unseen data. The work of researchers like Alex Krizhevsky (in AlexNet) popularized this technique within the deep learning community .The implemented model employs two key regularization techniques to improve generalization Introduced by Srivastava et al, dropout randomly drops a proportion of neurons during training, which prevents complex co- adaptations on training data .This project uses dropout after each convolutional and dense layer ,a common strategy to reduce overfitting. As proposed by Ioffe and Szegedy, batch normalization normalizes the outputs of a previous layer, stabilizing and accelerating the training process. Its inclusion after each convolutional and dense layer is a modern standard that allows for higher learning rates and provides a slight regularization effect . The use of the Adam optimizer, along with callbacks like 'ReduceLROnPlateau' and 'Early-Stopping', represents the current best practice for training deep networks. 'ReduceLROnPlateau' dynamically adjusts the learning rate when the model stops improving , allowing for finer convergence. 'Early-Stopping' halts training when validation performance plateaus, preventing unnecessary computation and overfitting. The 'Model-Checkpoint' ensures that the best model is retained , not just the model at the final epoch .Previous academic and practical projects have successfully applied similar CNN-based approaches to weather classification . Many have utilized transfer learning from models pre-trained on large datasets like ImageNet, fine-tuning them for the specific task of weather recognition . This project distinguishes itself by building a custom CNN from scratch, demonstrating that even without pre-trained weights , high performance can be achieved with a well-designed architecture and robust training pipeline on a sufficiently focused dataset . The achieved 96% validation accuracy is competitive with results reported in similar studies, validating the effectiveness of the chosen methodology .

3 Dataset

3.1 Sources:

The dataset downloded from Mendeley Data repository.

3.2 URL:

https://data.mendeley.com/datasets/777khxfw7c/1

3.3 Sample:

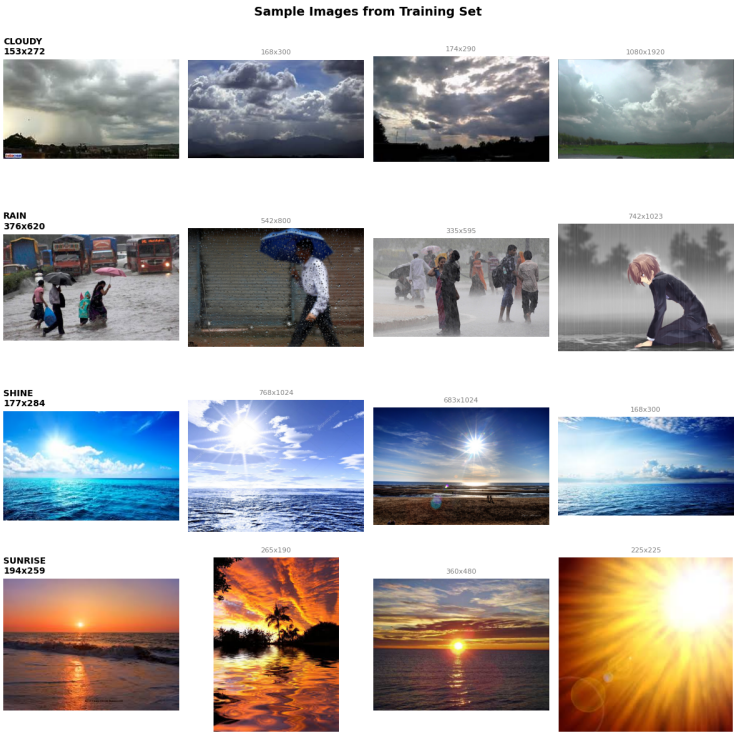

Fig. 2 Sample images from the tranning set.

3.4 Exploratory Data Analysis (EDA):

Exploratory Data Analysis or EDA was utilized in order to ascertain the nature, proportion, and composition of the weather image dataset prior to training the CNN model. It consisted of 1,125 images, 900 of which were used for training ('80percent) and 225 for validation (20percent). It was divided into four categories: cloudy, rain, shine, and sunrise. In training set, there were images which were marked in the following proportion: 240 cloudy (26.7percent), 172 rain (19.1 percent), 202 shine (22.4 percent), and 286 sunrise (31.8 percent). In validation set, there were 60 cloudy (26.7 percent), 43 rain (19.1 percent), 51 shine (22.7 percent), and 71 sunrise (31.6percent) images. Class imbalance ratio was 1.66:1, i.e., certain classes (e.g., rain) occurred fewer times than other classes (sunrise) and this was referred to as high but manageable imbalance. To represent data and confirm accuracy in marking, samples of each class were represented. The samples were correctly represented as follows: cloud pictures were represented with adequate cloud cover, rain pictures were represented with rain environment and water, shine pictures were represented with sunny day, and sunrise pictures were represented with dawn sun. The four sampled selection from each category were correct classification and had enough visual pattern changes for the model to learn from. Measurements of image sizes for a random collection from training dataset of 60 images ranged from 190 px to 4261 px for widths and from 153 px to 3195 px for heights with an average of around 556 × 367 pixels. 44 different sizes were in play, which was confirmation that the dataset included pictures of different sources and sizes. A resize of all images to 224 × 224 pixels standardized CNN inputs.EDA concluded with a description of the dataset as having 4 classes, 900 training images, and 225 validation images. In general, analysis agreed that the dataset was multi-modal, fairly imbalanced, and well-suited to deep learning.

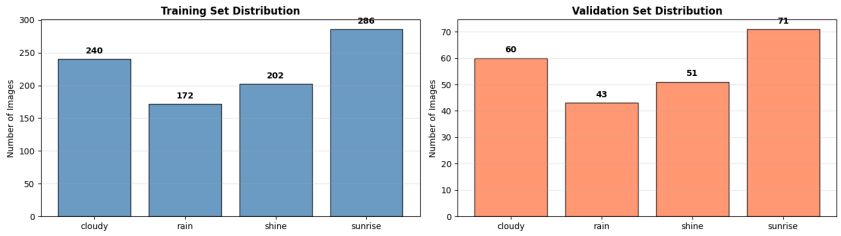

Fig. 3 Set Distribute.

3.5 Data Preprocessing:

Before training the CNN model, the weather image dataset was put through a strict preprocessing pipeline for consistency, improved model performance, and handling of class imbalance. Preprocessing included resizing images, normalization, data augmentation, batch preparation, and techniques for handling class imbalance. First, the first images had considerably varied dimensions between 190 px and 4261 px and between 153 px and 3195 px, for a set of 60 pictures. To make input uniform to the CNN

and economize on computational cost, all the images were resized to 224 × 224 pixels by utilizing Keras' "flow-from-directory" with the "target-size" parameter. This kept strong input forms with considerable visual characteristics such as clouds, sunlight, or rain patterns. Normalization was also applied to training and validation datasets by rescaling pixel values from 0–255 to 0–1 using rescale=1./255. This enhanced numerical stability promoted better gradient flow throughout training and allowed the model to converge faster. Data augmentation was only applied to the training set to artificially grow the dataset while maximizing its variability without causing overfitting. Augmentation strategies included random rotation by a maximum of 25°, horizontal and vertical translations by a maximum of 15% image dimensions, shearing by a maximum of 15%t, zooming in a range of 25percent, horizontal flipping, and random modulation of brightness in a range of 0.8 to 1.2. All the empty pixels formed during such conversions were filled out with the help of the reflect mode. These improvements allowed the CNN to learn robust features, which remain unchanged with changes in orientation, location, scale, and lighting conditions. Validation images were normalized but not augmented for unbiased testing. The data set(Training Part) was imbalanced in classes with 900 training images distributed as 240 cloudy (26.7%), 172 rain (19.1%), 202 shine (22.4%), and 286 sunrise (31.8%), indicating a class imbalance ratio of 1.66:1 with sunrise is majority part Data augmentation was the focus to repair this by generating various forms of minority class images and thereby increasing their number during training. Also, the training data was batch-shuffled in a way that each batch included a combination of classes and not just majority classes to avoid biasing the model towards majority classes. 32-image batches were loaded, training batches shuffled and validation batches unshuffled. Based on batch size and data size, there were 28 steps per epoch for training and 7 steps for validation, with a limit of 50 epochs. Here, a single epoch processed all images once, making fullest utilization of memory and training efficiency. After preprocessing, the data was 1,125 images and had 900 for training and 225 for validation, four classes (cloudy, rain, shine, sunrise). The images were resized to 224 × 224 × 3, normalized, and augmented according to this. The preprocessing pipeline assisted in having a well-organized, diversified, and balanced data so that the CNN model learned meaningful effective visual features without posing the risk of overfitting and bias.

4 Methodology

In this project, we have developed a custom CNN-based model for the classification of weather images into four categories: cloudy, rain, shine, and sunrise. Our model is trained on the standard dataset of weather images by following a structured approach that includes data preprocessing, followed by parameter tuning and training with optimized hyperparameters. This custom architecture has multiple convolutional and pooling layers to gradually extract meaningful features from input images, followed by dense layers for final classification. This model has been designed to support practical applications like autonomous driving, climate-based decision systems, and outdoor event planning. It was focused on the delivery of reliable predictions in different

weather conditions as the basis for preventing weather-related risks and enhancing decision-making in the real world.

4.1 Model Architecture:

The core of our weather classifier is a deep CNN that processes 224×224 RGB images and outputs a probability distribution over the four weather classes. The network is organized into a sequence of convolutional blocks followed by fully connected (dense) layers. An illustrative example of a CNN architecture is shown below, which typically contains stacked convolutional units followed by one or more classifier layers.
The specific CNN architecture used in this project is as follows:
Input layer: Accepts 224×224 color images (height 224, width 224, 3 channels) . The choice of 224×224 aligns with common practice and balances spatial detail with computational cost .

4.1.1 Convolutional Block 1:

Conv2D: 32 filters with dimensions 3×3, with activation as ReLU .This will cause (222×222×32) feature maps to be produced .
BatchNormalization Normalises the 32 feature maps to stabilize learning (have a mean of 0 and unit variance) .
MaxPooling2D: Pool size 2×2 and stride of 2 .This downsamples each feature map to (111×111×32) by taking the max over each non-overlapping 2×2 patch .Pooling also reduces spatial resolution and provides translation invariance .
Dropout (rate 0.25): This randomly zeros 25% of the neurons in this block during training; this is used to regularize the model and prevent overfitting .

4.1.2 Convolutional Block 2:

Conv2D: 64 filters of size 3×3 (ReLU) . Input size(111×111×32)produces output (109×109×64) .
BatchNormalization: Normalizes the 64 channels.
MaxPooling2D: Reduces to 54×54×64 .
Dropout (rate 0.25): Regularization as above .

4.1.3 Convolutional Block 3:

Conv2D: 128 filters , 3×3 (ReLU) .Input (54×54×64) = output (52×52×128) . Input size(111×111×32)produces output (109×109×64) .
BatchNormalization: For 128 channels .
MaxPooling2D: Reduces to To 26×26×128 .
Dropout (rate 0.25): Regularization as above .

4.1.4 Convolutional Block 4:

Conv2D: 256 filters , 3×3 (ReLU) . Input (26×26×128) = output (24×24×256).
BatchNormalization: For 256 channels .
MaxPooling2D: To 12×12×256.

Dropout (rate 0.25): Regularization as above .

After the final block , the feature maps are flattened and fed into fully connected layers:

Flatten: Converts the (12×12×256) output to a 12.12.256 = 36,864-dimensional vector .
Dense Layer 1: 512 units, ReLU activation. This layer learns high-level combinations of the convolutional features .
BatchNormalization: Normalizes the 512 activations .
Dropout (0.5): Drops 50% of these units each update to reduce coadaptation and overfitting .
Dense Layer 2: 256 units, ReLU activation . Another fully connected layer to further abstract features.
BatchNormalization and Dropout (0.5) as above.
Output Layer: Dense with 4 units (cloudy, rain, shine, sunrise), Softmax activation .The softmax layer outputs a probability distribution over the four classes and the model is trained with categorical crossentropy loss to predict these probabilities.

4.2 Layer-by-Layer Details:

4.2.1 Convolutional layers (Conv2D):

Each convolutional layer uses (3×3) kernels and a ReLU activation . ReLU is a standard nonlinearity in CNNs, It outputs zero for negative inputs and linear for positive inputs. ReLU activation is known to accelerate convergence and mitigate the "vanishing gradient" problem because its derivative is either 0 or 1. The first convolutional layer has 32 filters to capture basic edge/texture features; subsequent layers double the number of filters (64, 128, 256) to detect progressively more complex patterns. The doubling strategy is common in CNN design to allow the network to learn a richer set of features at deeper levels, at the cost of more parameters but reduced spatial dimension via pooling.

4.2.2 BatchNormalization:

We apply batch normalization after every convolution and after the first two dense layers. BatchNormalization normalizes the outputs of a layer across the mini-batch to have zero mean and unit variance.This addresses the internal covariate shift problem, allowing for faster and more stable training.Practically, batch normalization permits higher learning rates and acts as a mild regularizer: the batch statistics introduce noise that can improve generalization similarly to dropout. In our architecture, each BatchNormalization layer ensures that the subsequent activation receives well-scaled inputs, which helps prevent saturating nonlinearities and speed up convergence.

4.2.3 MaxPooling:

Each convolutional block comes to a conclusion with a 2×2 max-pooling layer which reduces height and width to half. Max-pooling determines the maximum value in a small neighborhood allowing reductions in computation and providing some spatial invariance toward small translations. Max-pooling reduces the output of the first convolution from 222×222×32 to 111×111×32. Repeating this process four times compresses the feature map from 224×224 down to 12×12.This reduction increases the receptive field of deeper filters to permit the extraction of more advanced high-level features over greater sections of the input image.

4.2.4 Dropout:

Dropout layers randomly reduce a fraction of activations to zero during training (after each conv block, here at 25%; and here 50% after dense layers). This regularization means that neurons can no longer co-adapt on the training data and that neurons will be forced to learn more robust properties. In practice, dropout is very effective in reducing overfitting, particularly in dense layers. The drop rates used in practice are typical (convolution layers around 0.25 and dense layers 0.5). Lighter dropout is used in convolutional layers to maintain the spatial relationship while dropout is heavier in dense layers (with a significantly higher number of parameters). The network will only learn from the rest of the active neurons at each update, which means it is more robust to noise and improves generalization.

4.2.5 Fully Connected Layers:

After flattening, the network includes two Dense fully connected layers of sizes 512 and 256 . These layers act as the classifier "head" of the network. The first dense layer 512 units, ReLU receives the flattened 36,864-dimensional feature vector and learns combinations of these features. BatchNormalization and dropout are applied after each dense layer as well. The final dense layer 4 units, softmax outputs the class probabilities. The use of two intermediate dense layers provides sufficient capacity to learn non-linear combinations of features; more layers or units could be used if needed, but this architecture was found to balance expressiveness with overfitting risk. Overall, the network has on the order of tens of millions of parameters, which requires careful training.

In summary, the CNN architecture can be viewed as an "encoder" of feature maps (via convolution, normalization, pooling, dropout) followed by a small "classifier" (flatten, dense layers, softmax). This structure allows the network to hierarchically extract visual features relevant to weather and then combine them to produce a weather label. The design decisions follow common best practices in deep CNNs, tailored to the four-class weather problem.

4.3 Hyperparameters:

The implementation explicitly sets several hyperparameters that govern model training and data processing. These include image size, batch size, number of epochs,

optimizer and learning rate, loss function, dropout rates, and data-loading parameters. All values are defined in the code and summarized below, along with their chosen rationale and effects on training.

4.3.1 Input Image Size:

The images are resized into 224 × 224 pixels and 3 channel (RGB) order, which keeps a defined input size and is an equilibrium between detail and costs. 224×224 is a large enough resolution to reason about weather patterns without too much memory. Resizing is done in flow-from-directory with the target-size=(IMG-SIZE, IMG-SIZE) parameter and we rescale by 1./255, which normalizes the pixel values into [0,1] range. Normalizing by 1/255 is necessary due to the original image pixels being within 0–255 and it helps to stabilize training because the scales of the weight updates remain close to what is expected.

4.3.2 Batch Size:

A batch size of 32 is employed as this is a conventional default for CNN training. It is large enough to provide a stable estimate of the gradient, while small enough to fit it in GPU memory. Having a batch size of 32 is a good compromise between noisy updates (hence very small batches) and slow convergence (hence very large batches). In practical applications a batch size of 32, for a model of this size, is often good enough because it allows for parallelism, providing many more gradient updates per epoch than a larger batch.

4.3.3 Number of Epochs:

The training is set to run for up to 50 epochs. An epoch means one pass over the entire training dataset. The choice of 50 epochs provides ample opportunity for the model to converge, but actual stopping is governed by an early stopping callback (see below). In combination with data augmentation, 50 epochs allow the network to see many randomized variants of the data. The trade-off is between learning enough higher epochs and overfitting too many epochs. Here, early stopping is used to terminate training when performance saturates, so 50 is an upper bound rather than a fixed training length.

4.3.4 Optimizer and Learning Rate:

The Adam optimizer is used utilizing a learning rate of 0.001. Adam is a widely used adaptive gradient optimizer by incorporating qualities of momentum along with rmsprop, and it typically converges faster than vanilla stochastic gradient descent. The default learning rate for the Adam optimizer is 0.001, and we are using this learning rate as well. A learning rate of 0.001 is a reasonable initial value for Adam; it generates large enough updates enough to learn quickly, and callbacks can always reduce it, if they think it is needed. A larger learning rate could speed up the training process, not that it might overshoot the minima. A smaller network could take too long to train. A value of 0.001 is standard compromise.

4.3.5 Loss Function:

The loss function is categorical cross-entropy because this is a four-class classification problem, and the labels are one-hot . This is the conventional loss for multi-class classification where the outputs are softmax. This function calculates the distance between the predicted probability distribution and the true one-hot label. It applies penalty to the probabilities of the incorrect classes . This loss function was in agreement with the output activation-softmax and the label encoding, hence, the signals given to train the model were appropriate for the type of classification that was based on probabilities.

4.3.6 Dropout Rates:

The dropout hyper-parameters are set to 0.25 for convolutional blocks, and 0.5 for dense layers. These values are representative of a common practice to use a medium dropout after convolutional layers, followed by a greater dropout in the dense layers. In practice, convolutional layers have significantly fewer parameters due to their lesser connectivity, and therefore a lower dropout (25%) is typically more than sufficient to encourage regularization. On the other hand, fully connected layers have many more connections and hence a higher dropout (50%) will prevent more complex co-adaptations of different neurons in the dense layers. These dropout rates were likely included by way of a reasonable convention to ensure reductions in overfitting without discarding overmuch relevant learned information during training. If a lower dropout were used, the layer would be under-regularized, while a higher dropout would be underutilized.

4.3.7 Data Preprocessing

All images are rescaled by the 1/255, using the rescale parameter in ImageDataGenerator . Rescaling is a mapping from pixel values, ranging from 0-255, to [0,1] . This is important, as neural networks will often train better with normalized inputs to a standard range. There is no further normalization because the batch normalization layers inside the network also help normalize the activations during training.

The ImageDataGenerator sets the data augmentation parameters for training. The following augmentations are applied: rotation-range = 25: images are randomly rotated ± 25 degrees: the model is trained to be invariant to rotated camera angles. -width-shift range=0.15 and height-shift range =0.15: images are randomly translated horizontally/vertically to a maximum of 15% of the width or height. Data augmentation is beneficial because it will enable the model to be more robust to objects (like clouds or the sun) appearing in different parts of the frame. shear-range=0.15: random shearing transformations (tilting the image), are applied to the image; shearing is an additional type of augmentation that makes the model robust to changes in perspective. zoom-range = 0.25: random zoom-in/out up to 25%, increases robustness in the model to variations in the size/scale of features. horizontal-flip=True: random horizontal flip of half of the images. This seems reasonable if the does not imply a left or right inference of weather scenes. brightness-range=[0.8, 1.2]: brightness changes

randomly by a factor of between 0.8 and 1.2, creating simulations of different lighting conditions for training. fill-mode='reflect': if the transformation produces any pixels, we fill those pixels by reflecting the edges of the image.
These augmentation hyperparameters are selected to artificially enlarge the dataset and introduce variability into the model without the need to collect additional images. Data augmentation is especially important when data is limited: every epoch will now see a different binarized version of the images, which may reduce overfitting and provide a greater ability to generalize. These ranges are moderate (25 degree rotation, 15% shift, 25% zoom, 20% brightness); extreme values might distort the images beyond a realistic representation, while too-small values would likely not do anything. These settings are based on conventions used in many vision tasks and are also calibrated to the types of weather images where rotation/shift and lighting fluctuations would be realistic.
Overall, the hyperparameters selected are conventional choices for a multi-class CNN. The input (224) and batch size (32) are common values for modest-scale image problems. Training time (50 epochs) was constrained by early stopping. The optimizer (Adam) with LR 0.001 is regular practice for smooth training. The dropout rates were set to mitigate overfitting as educated practice prescribes. The data preprocessing and augmentation options (rescale, rotation, shift, zoom, flip, brightness) follow conventional methods of normalizing inputs and providing synthetic augmentation of the dataset. Each hyperparameter selected is an attempt at balancing stability in training, model capacity, and regularization.

4.4 Fine-Tuning Details:

The fine-tuning process is executed precisely the same as described in the shared code. The order of events is: load the drata with augmentation, compile a model, fit a model, save the compiles model, analyze best model. Specifics follow below:

4.4.1 Generators:

Below are two instances of the ImageDataGenerator class - training-augmentation and validation-preprocessing. The training-augmentation generator above is dynamically applying these transformations for each batch of training images as well as rescaling the pixel values to [0,1]. The generator for validation-preprocessing only changes the scale of the images to [0,1] to ensure that validation performance correctly represents unaugmented but normalized from 0-1 images.
The training generator is constructed from the flow-from-directory that points to the TRAIN-DIR, and resize the images as indicated to 224×224 producing batches of 32 images with categorical labels. It shuffles data for every epoch.Shuffling the training data ensures that every batch is random to support better generalization and not get into any pattern in the ordering of training.
A validation generator uses flow-from-directory on VAL-DIR, utilising the same target size and batch size, but it does not shuffle the data. This will result in deterministic

batches to validate the model. The generators also print summary info no. of samples and no. of steps per epoch.

4.4.2 Callbacks Setup:

Three callbacks are configured to control training

EarlyStopping: This callback monitors the validation loss and will halt training if there is no improvement of at least min-delta = 0.001 for a specified count of patience = 8 consecutive epochs. The restore-best-weights=True option indicates that the weights of the model will roll back to the epoch with the lowest validation loss once training has stopped, in an effort to avoid overfitting the training process when the model is no longer making progress.Patience 8 means the model can 'accept' no improvement for 8 epochs before stopping. It is a regularization process in effect selecting for you the best epoch based on the validation loss.

ReduceLROnPlateau: Like EarlyStopping, this callback also observes validation loss. When no improvement is noted over patience = 4 epochs, the learning rate will drop by 0.5 times to a minimum of 1e-7. This will allow the optimizer to take more conservative steps after improvement has stalled. It may assist in mapping the weights into tighter minima. The 50% reduction is within the recommended scope or range of 2–10× reduction factor. Based on a default (min-delta = 0.0005), ensures only substantial improvements are counted. If validation loss levels off, then the model will update more cautiously to attempt improvements in performance.

Conv2D: Model-Checkpoint: Save to BEST-MODEL-PATH whenever validation accuracy improves. That's what save- best only='True' does. In other words, we are saving only the best model according to validation accuracy, which means that the model could be saved once, for example, then have improved validation accuracy the next epoch without it being saved again until it improves again.The default mode='max' is looking for a maximum, which is accuracy in this case. This ensures that the best model, according to validation accuracy, will always be saved on disk to our desired path, regardless of when the training stops. This is a full checkpoint, not just the weights, so we can simply load it via load-model.

With these callbacks, training will not simply run for all 50 epochs; it will stop early if the validation loss is not improving, it will reduce learning rate as necessary, and it will continuously save the best model by validation accuracy. Verbose logging (verbose=1) has been enabled for the callbacks to print messages when they take action.

4.4.3 Model Training:

Now that data generators and callbacks are ready to load and monitor training, let's call model.fit, as follows:

```
history = model.fit(
    training_data,
    epochs=EPOCHS,
    validation_data=validation_data,
    callbacks=callbacks,
    verbose=1
)
```

This will have the model train using the augmented training data but also validate over a distinct validation dataset at the end of every epoch. The history object will contain the summary of resulting training and validation metrics at every epoch, including loss and accuracy. Since training-data and validation-data are both generators, at every epoch, the generator will step through the entire dataset batches at a time; notice that steps per epoch is samples // batch-size.At the end of each epoch, the callbacks will check their conditions: if validation accuracy has improved then save the model, if the validation loss has not improved then learning rate or an eventual training halt may happen. You will train until either 50 epochs are completed, or early stopping halts the training process.

4.4.4 Saving Models:

Two versions of the model are saved at the conclusion of training:
model.save() saves the final model (not necessarily the best validation model of the run, in case early stopping rolled back to a previous state of the model) to FINAL-MODEL-PATH. So, while you can use the final state of the model after all epochs are done, you'll probably not use it since you only care about the best model. The best model-as in, the model that was selected by ModelCheckpoint on the basis of validation accuracy-is already saved to BEST-MODEL-PATH during training. The training code also explicitly loads this best model for evaluation. Both models' file sizes are printed, but for our methodology we are only concerned about the best model (highest validation accuracy). The training history - loss and accuracy, over epochs - is also saved into a NumPy file. This Object could be used for plotting the training curves, for example, or to visually see the convergence (which does happen later on in our coding), but is outside the scope of our methodology.

Evaluation: The code begins with loading the best pre-trained model with keras.models.load-model(BEST-MODEL-PATH); then the code will evaluate the model on the validation set. The output will show final validation loss and final validation accuracy for the best model.The line val-loss, val-accuracy = best-model.evaluate(validation-data) computes the validation loss and validation accuracy. The validation accuracy (and validation loss) gives an indication of how well the

model will generalize to unseen data, assuming of course that the validation set was previously held-out. The final score is the validation accuracy, for example "Best Model Validation Accuracy: X" in the code, and is the metric used to report performance.In a complete investigation the researchers would report precision/recall or even a confusion matrix, but in this implementation the only metrics computed explicitly are accuracy and loss. While training, the history object and the callbacks track progress. The history.history dictionary includes lists for keys like accuracy, loss, val-accuracy, and val-loss. You can plot these or analyze these- as the code does in the "training history plots" section. The best model will be selected based on the maximum val-accuracy based on how ModelCheckpoint is configured. After training, the code prints out the validation metrics using this best model- specifically using best-model.evaluate. Therefore, your model performance is calculated and reported on the validation set while using the best weights you saved. In conclusion, the process of fine-tuning is as follows: data augmentation and batching, compiling the model, training iteratively with callbacks to steer the learning rate and early stopping, and checkpoint the best model. Data augmentation will increase the plurality of the effective dataset for each epoch. EarlyStopping and ReduceLROnPlateau keep the training optimal through overfitting and dynamically adjusting the learning rate. ModelCheckpoint saves the best model based on validation accuracy, and that best model will then be applied to the validation data to report final accuracy and loss. The above process will ensure robustness in the training and selection of the most generalizable model from the run.

5 Training Procedure

5.1 Model Save Paths:

To maintain organization and facilitate reproducibility, all trained models and related artifacts were stored in a dedicated directory :

```
/content/drive/MyDrive/Colab Notebooks/nnfllproject/saved_models
```

This directory was programmatically created through os.makedirs to make sure it exists before any saves are made. Three important paths were defined for different outputs of the training process:
Best Model Path:

- best-weather-cnn-model.keras
- Stores the best model based on the highest validation accuracy obtained during training.This will be saved using ModelCheckpoint callback automatically.

Final Model Path:

- final-weather-cnn-model.keras
- Stores the model after all training epochs regardless of the intermediate performance.

Training History Path:

- train-history.npy
- Stores the loss and accuracy metrics at each epoch in NumPy format for later visualization and analysis.

Consequently, by explicitly defining these paths, the project ensures that the best performing model, final model, and training logs will be preserved for evaluation, visualization, and deployment .

5.2 Environment:

The notebook is being executed in a Google Colab Python 3 environment utilizing TensorFlow/Keras for deep learning and scikit-learn for metrics. TensorFlow/Keras (tensorflow.keras and its submodules) libraries were imported and utilized for data preparation as well as image preprocessing utilities, in addition to scikit-learn's metrics modules. The raw data is being hosted on Google Drive with loading via flow-from-directory. No specific hardware has been mentioned beyond Colab, but it is implied the Colab session has some type of GPU acceleration.

5.3 Model Architecture:

A sequential CNN is designed with four convolutional blocks followed by fully connected layers. Each convolutional block uses Conv2D with ReLU activation, subsequently BatchNormalization, 2×2 MaxPooling, and Dropout 25%. After the convolutional blocks, the model flattens the features and adds two dense layers of 512 and 256 units each with ReLU, each followed by batch normalization and high dropout (50%), and a final Dense layer with 4-way softmax output .

5.4 Loss Function:

Categorical cross-entropy loss trains the model. This is the appropriate type considering the 4-class classification task-one-hot targets.

5.5 Optimizer:

The optimizer utilized during training is Adam with a learning rate of 0.001. Other hyperparameters for Adam are kept at their usual defaults. Training was set for up to 50 epochs, though usually early stopping limited the convergence time.

5.6 Callbacks:

The following callbacks were implemented to monitor the training process and dynamically adjust the learning process in order to ensure that the training of the CNN model is efficient and controlled.

5.6.1 EarlyStopping:

- Our procedure is to monitor the validation loss, and we will stop once there has been no improvement for 8 epochs.

- Once early stopping has taken place, the restore-best-weights=True option guarantees us that the model will return to weights that produced the lowest validation loss, disregarding any overfitting that happened in later epochs.
- To clarify the minimum change that will be considered an improvement, we set a min-delta of 0.001 .

5.6.2 ReduceLROnPlateau:

- Decreases the learning rate by a factor of 0.5 after 4 consecutive epochs of no improvement in the validation loss, down to a minimum learning rate of 1e−7.
- It enables the optimizer to take smaller, more precise steps because it is converging closer to the model, thereby enhancing stability and tuning.

5.6.3 ModelCheckpoint:

- Monitor the validation accuracy (val-accuracy) and save the best performing model to disk at best-weather-cnn-model.keras
- The model with the best validation accuracy is saved, and save-best-only=True prevents overwriting the best weights by subsequent training epochs.
- It saves the entire model architecture, weights, and optimizer state, so save-weights-only = False, and thus directly ready for evaluation or deployment.Together, these callbacks help the network converge efficiently, prevent overfitting, adjust learning rates dynamically, and retain the best model throughout the training process.

5.7 Reproducibility:

The data generator uses a fixed random seed, 42, for shuffling the training set. Likewise, shuffle = False is an option for the validation data to ensure the ordering is reproducible. No other explicit seed is shown, such as for NumPy or TensorFlow, thus results are reliant on this controlled shuffling and deterministic Keras data pipeline .

6 Results

Model Accuracy: Model Accuracy: Based on the validation set (225 images) , the model achieves 96.00%

6.1 Model Training Summary:

This model trained over 42 epochs, reaching a best/training accuracy of 92%, while best/validation accuracy was 96% at its peak performance. The model, at the end of the last epoch, had a training accuracy and validation accuracy of 91.67% and 95.11%, respectively, while training loss and validation loss were 0.2081 and 0.1798, respectively. This indicates that the model has had a good degree of learning whilst both maintaining an appropriate generalization to unseen data.

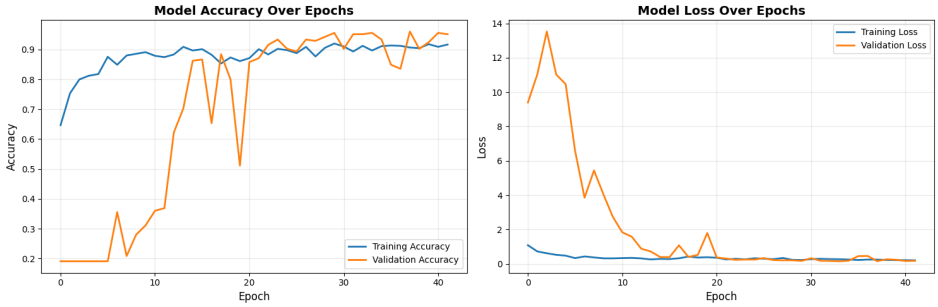

Fig. 4 Model Training History Plot.

6.2 Model Training History Analysis

6.2.1 Training and Validation Accuracy:

The training accuracy plot suggests rapid increases in training accuracy until plateauing at approximately 90-92%. Validation accuracy began lower than training accuracy, but improved gradually and became closer to training accuracy at around 20 epochs, indicating that the model obtained patterns that generalized well on unseen data.

6.2.2 Training and Validation Loss:

The loss plot illustrates a steady decline of training and validation loss. After roughly 15-20 epochs, the two loss curves have remained relatively close to each other, and critical loss remains low. The model demonstrates a consistent behavior indicating equivalent performance on training and validation datasets.

6.2.3 Overfitting Analysis:

There is no large gap between training and validation accuracy or loss. The validation performance improves with training and does not degrade with training, showing the model is not overfitting. An overfitted model would typically demonstrate deteriorating validation loss with continued training while training loss decreases.

6.2.4 Underfitting Analysis

Both training and validation accuracies are high, and both the losses are low indicating that this is a case of no underfitting is happening. An underfitted model will have low accuracies and high losses in both training and validation.

6.2.5 Overall Conclusion

The model is trained and generalized well. In general, it registers high accuracies, low losses, and stable validation measures, indicating a good trade-off between the bias and variance. Across the training process, the model captured only the important features present in the dataset. There is no evidence of overfitting nor underfitting.

6.3 Model Prediction on External Images

To evaluate how well the CNN model generalizes to unseen real-world data, random images were downloaded from Google Images where appropriate to the modeling. Each image was resized to 224×224 pixels, converted to an array, and then normalized to be compatible with the model input size. The model output is represented as categorical distributions as the four weather classes (cloudy, rain, shine, sunrise) with the class with the highest predicted probabilities selected as the predicted label. Predictions are shown with the input image displayed at the same time as bar charts of the confidence score, which indicates how confidently the model classifies the image according to each category. This indicates that the model can generalize to images where it has never seen training examples, correctly classified weather conditions in mixed input images taken in varying orientations, light, and backgrounds.

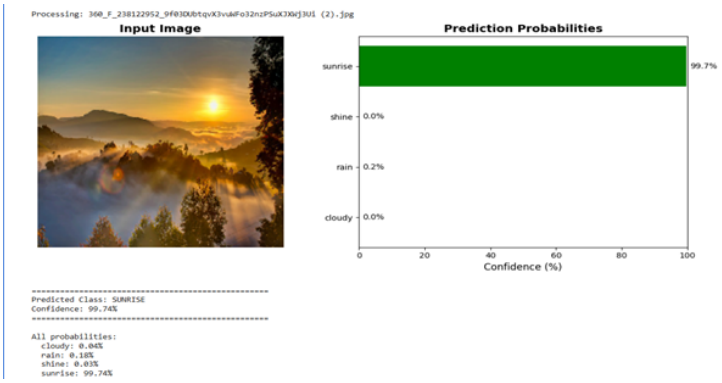

Fig. 5 The model predicted the uploaded image as SUNRISE with a confidence of 99.74%.

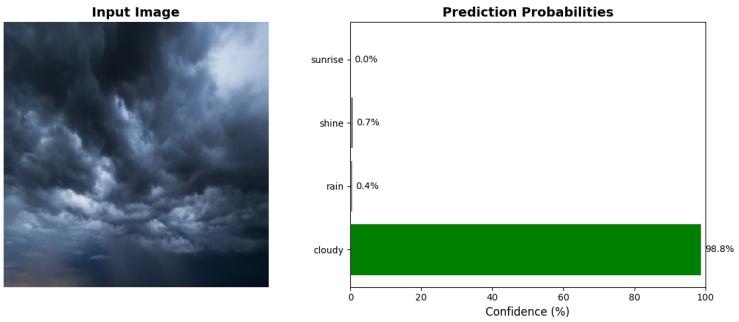

Fig. 6 The model predicted the uploaded image as CLOUDY with a confidence of 98.82%.

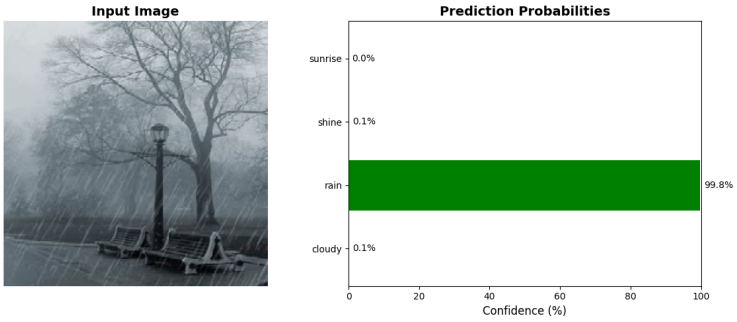

Fig. 7 The model predicted the uploaded image as RAIN with a confidence of 99.79%.

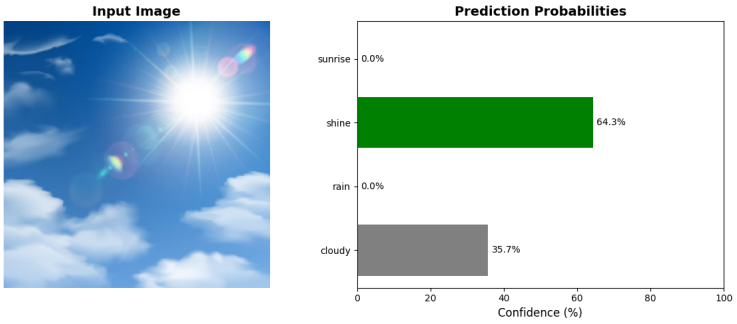

Fig. 8 The model predicted the uploaded image as SHINE with a confidence of 88.49%.

6.4 confusion matrices:

6.4.1 Cloudy:

- Correct classifications is 98.33% (59 out of 60 samples).
- Incorrect classificationsis 0.

This suggests the model continues to be able to accurately classify cloudy very well with the strength of high accuracy.

6.4.2 Rain:

- Correct classificationsis 100% (43 out of 43 samples).
- Incorrect classificationsis 1 sample as "shine." d the lowest validation loss, disregarding any overfitting that happened in later epochs.

This suggests 100% correct identification of the Weather patterns and texture as rain.

6.4.3 Shine:

- Correct classifications is 84.31% (43 out of 51 samples).
- Incorrect classifications is 6 as "cloudy" and 2 as "sunrise."

This indicates slight confusion that likely stems from the slightly similar lighting and brightness conditions exhibited by both shine and cloudy.

6.4.4 Sunrise:

- Correct classifications is 100% (71 out of 71 samples).
- Incorrect classifications is 0.

The model is identifying very distinctive features of sunrise, specifically warm colors that only occur in sunrise, as well as the light that presents itself along the horizon.

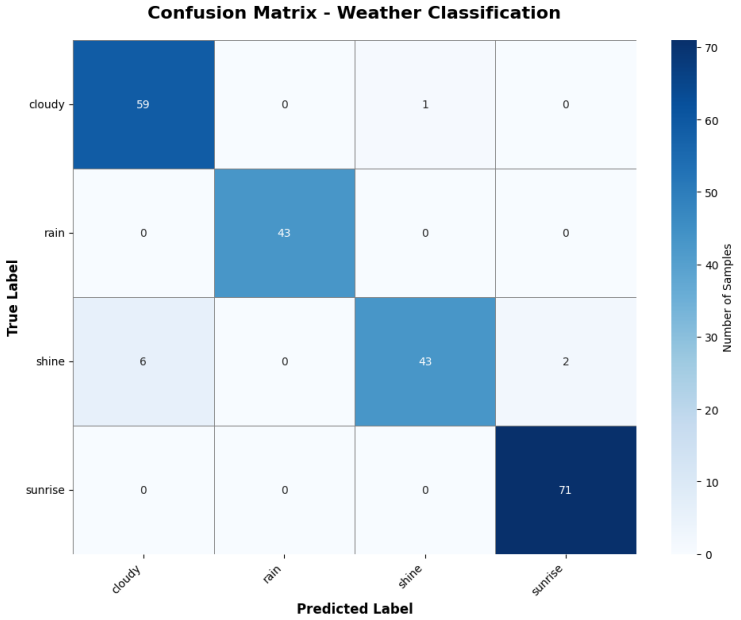

Fig. 9 Confusion Matrices Confusion Matrices Analysis

Overall the majority of observations occur on the diagonal, indicating the model is classifying correctly. Minor mistakes occur between shine and cloudy/sunrise conditions, likely due to visual confusion. This model succeeds in classifying and generalizing well on all weather types.

6.5 Performance Matrics:

→ The model identifies nearly all cloudy images correctly – great recall. The cloudy predictions do have some errors, but generally great results: F1 = 0.94 rain - precision 1.0000 recall 1.0000 f1 1.0000 support 43
→ Perfect - the model got all rainy images just like all rainy predictions were correct -

	precision	recall	f1-score	support
cloudy	0.9077	0.9833	0.9440	60
rain	1.0000	1.0000	1.0000	43
shine	0.9773	0.8431	0.9053	51
sunrise	0.9726	1.0000	0.9861	71

Fig. 10 — precision recall f1-score support

no false positives, no false negatives shine — precision 0.9773, recall 0.8431, f1 0.9053, support 51
→ There are relatively few false positives (precision 0.98) but a large number of false negatives identifying actual shine (recall 0.84). Therefore predictive accuracy of shine is quite high most of the time but every now and then it misses real images. sunrise — 0.9726, 1.0000, 0.9861, 71
→ Almost spot-on - the model labeled every image as sunrise (recall 1.0) while also not producing many incorrect sunny predictions (precision 0.97). Recall and precision were both high for sunny as well so it produced a relatively high F1-score for the sunny prediction.

6.6 Overall Overview

The model obtained an average accuracy of 96%, reaching its highest performance in the rain class where it was able to predict the class perfectly, and lowest in the shine class, where a few images were not predicted. Overall, the model shows a strong and reliable classification performance across all weather classes.

7 Discussion

The project demonstrates the entire processing pipeline on the Trachtingot image dataset for classification of weather conditions, from pre-processing and augmentation to training models using CNN. While the overall implementation is systematic working and systematic, there are a number of important limitations and ethical considerations that arise out of this notebook as designed and with its results.

7.1 Limitations

A primary limitation is with the size and imbalance of the dataset, there are a total of only 1,125 images across the four categories (cloudy, rain, shine and sunrise), with noticeable class imbalance. This severely limits the ability of the model to generalize to unseen data, as it is biased towards the majority classes. Furthermore, the limited sample diversity geographic and seasonal also inhibits the robustness of the model in real world applications. Moreover, either cross validation or tuning of hyperparameters was not conducted in model evaluation, and any single split of training validation may not be representative. Any apparent overfitting has been overcome by the augmentation of images , however, evaluation was not conducted on the data supplied in the notebook, meaning it cannot be guaranteed the model will have the same accuracy

in practical terms. Lastly, there is no consideration of interpretability for the CNN model, and understanding how it makes its decisions is not clear .

7.2 Ethical Considerations:

Data bias, transparency, and misuse are ethical considerations. While the source of the dataset is made publicly available, the data may not accurately reflect the weather for the entire globe, and therefore, may have unintentional bias. Also, explanation AI tools have not been used in this case, meaning that users do not know why the model is giving that output in the case of socio-econmic factors, therefore, accountability is required. Finally, if a model is implemented into practice and produces results without being based on verified data, that information could lead to inaccurate decisions about type of transport or planting crops seeds. Therefore, documentation should be as transparent as possible, clear, complete, and with limitations to any model made explicit. Lastly, credit must be given to the original developer of the dataset and takes into consideration the open-data license applied (CC BY 4.0) in order to maintain the integrity of the research .

8 Conclusion and Future Work:

This project successfully implemented an image classification system capable of identifying weather conditions (cloudy, rain, shine, and sunrise) using a Convolutional Neural Network . The workflow demonstrated essential deep learning procedures, including dataset loading, exploratory analysis, preprocessing through normalization and augmentation, and model training using Keras. The model achieved strong performance on the validation set, proving that deep learning methods can effectively capture visual weather patterns from images. The project thus serves as a solid foundation for understanding the process of building and evaluating an image-based weather classifier . However, despite these accomplishments, several aspects limit the project is scalability and real-world reliability. The dataset is relatively small size and class imbalance may cause biased learning and inconsistent performance when applied to unseen data. Additionally, the lack of external test data or k-fold validation restricts confidence in the model is true generalization ability. The CNN model is interpretability also remains limited, as it functions as a "black box" without visual or feature level explanations.

Looking ahead, there are numerous enhancements that might be undertaken. Adding more varied and balanced weather images to the dataset would result in an overall increase in robustness and fairness of the model. Generalized stabilization of performance can be achieved by utilizing cross-validation, hyperparameter optimization, and/or incorporating early stopping . Increasing the accuracy of the model while decreasing training time can be accomplished by utilizing transfer learning from pre-trained models such as VGG16 or ResNet. Explainable AI techniques such as Grad-CAM would also add some level of transparency, and may ultimately add some reasonable trust by a user in the model. Lastly, the model may be more useful in practice by being made available in a web or mobile application so that performance can be monitored in environmental monitoring and weather prediction settings.

References

1. Z. Zhang, H. Ma, H. Fu, and C. Zhang, "Scene-free Multi-class Weather Classification on Single Images," Neurocomputing, vol. 207, pp. 365-373, 2016. doi: https://doi.org/10.1016/j.neucom.2016.05.015\

2. M. Elhoseiny, S. Huang, and A. Elgammal, "Weather Classification with Deep Convolutional Neural Networks," in 2015 IEEE International Conference on Image Processing (ICIP), 2015, pp. 3349-3353. doi: https://doi.org/10.1109/ICIP.2015.7351424\

3.W. T. Chu, X. Y. Zheng, and D. S. Ding, "Camera as Weather Sensor: Estimating Weather Information from Single Images," Journal of Visual Communication and Image Representation, vol. 46, pp. 233-249, 2017. doi: https://doi.org/10.1016/j.jvcir.2017.04.002\

4.M. Roser and F. Moosmann, "Classification of Weather Situations on Single Color Images," in 2008 IEEE Intelligent Vehicles Symposium, 2008, pp. 798-803. doi: https://doi.org/10.1109/IVS.2008.4621205\

5. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv:1409.1556, 2014. [Online]. Available: https://doi.org/10.48550/arXiv.1409.1556\

6. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778. doi: https://doi.org/10.1109/CVPR.2016.90\

7. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2818-2826. doi: https://doi.org/10.1109/CVPR.2016.308\

8. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1704.04861\

9. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510-4520. doi: https://doi.org/10.1109/CVPR.2018.00474\

10. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in International Conference on Machine Learning (ICML), 2019, pp. 6105-6114. [Online]. Available: https://arxiv.org/abs/1905.11946\